# `filter()`, `map()`, and `reduce()` in Python

## Functional Programming in Python

Python supports a **functional programming style**, where **functions can be passed as arguments** to other functions.
The three most commonly used higher-order functions are:

- `filter()`
- `map()`
- `reduce()`

## `filter()` Function

## Purpose:

Filters elements of a sequence based on a condition (returns only those elements that satisfy the condition).

```
filter(function, iterable)
```

- `function`: A function that returns `True` or `False`
- `iterable`: A sequence (like list, tuple, etc.)

**Returns an iterator** – wrap it in `list()` to get final results.

### Example:

```
nums = [1, 2, 3, 4, 5, 6]
even_nums = list(filter(lambda x: x % 2 == 0, nums))
print(even_nums)   # Output: [2, 4, 6]
```

Explanation:

- The lambda function checks for even numbers.
- `filter()` keeps only values for which function returns `True`.

### Without Lambda (using `def`):

```
def is_even(n):
    return n % 2 == 0

even_nums = list(filter(is_even, nums))
```

# `map()` Function

### Purpose:

Applies a function to **each item** in an iterable and returns a new sequence.

### Syntax:

```
map(function, iterable)
```

Returns an iterator – use `list()` to see the result.

### Example:

```
nums = [1, 2, 3, 4]
Data = list(map(lambda x: x+1, nums))
print(Data)   # Output: [2,3,4,5]
```

Explanation:

- The lambda returns `x increased by 1`
- `map()` applies it to every element of `nums`

### Without Lambda:

```
def increase(x):
    return x + 1

Data = list(map(increase, nums))
```

# `reduce()` Function

### Purpose:

Applies a function cumulatively to reduce the iterable to a **single value**.

### Syntax:

```
from functools import reduce

reduce(function, iterable)
```

- The function takes **two arguments**
- Applies cumulatively: `f(f(f(x1, x2), x3), x4)...`

### Example:

```
from functools import reduce

nums = [1, 2, 3, 4]
Sum = reduce(lambda x, y: x + y, nums)
print(Sum)   # Output: 10
```

Explanation:

- `(1+2) = 3, (2+3) = 5, (6+4) = 10`

**Without Lambda:**

```
def Add(x, y):
    return x + y

product = reduce(Add, nums)
```

# Comparison Table:

| Function | Purpose | Returns | Requires Import |
|---|---|---|---|
| `filter()` | Filters items based on condition | Filtered items | No |
| `map()` | Applies a function to each item | Transformed list | No |
| `reduce()` | Reduces to a single value | Single result | Yes (`functools`) |